Binary Search Tree

Saptarshi Pyne, Assistant Professor
Department of Computational and Data Sciences (CDS)
Indian Institute of Science Education and Research (IISER) Kolkata
Mohanpur, West Bengal, India 741246

CS3201 Programming and Data Structures II
March 11, 2025

Binary Search Tree (BST)

A BST is a binary tree where each node honours ‘the BST property’.

The BST Property

Let x be a node in a BST.

If y is a node in the left subtree of x, then y.key ≤ x.key.

Else if y is a node in the right subtree of x, then y.key ≥ x.key.

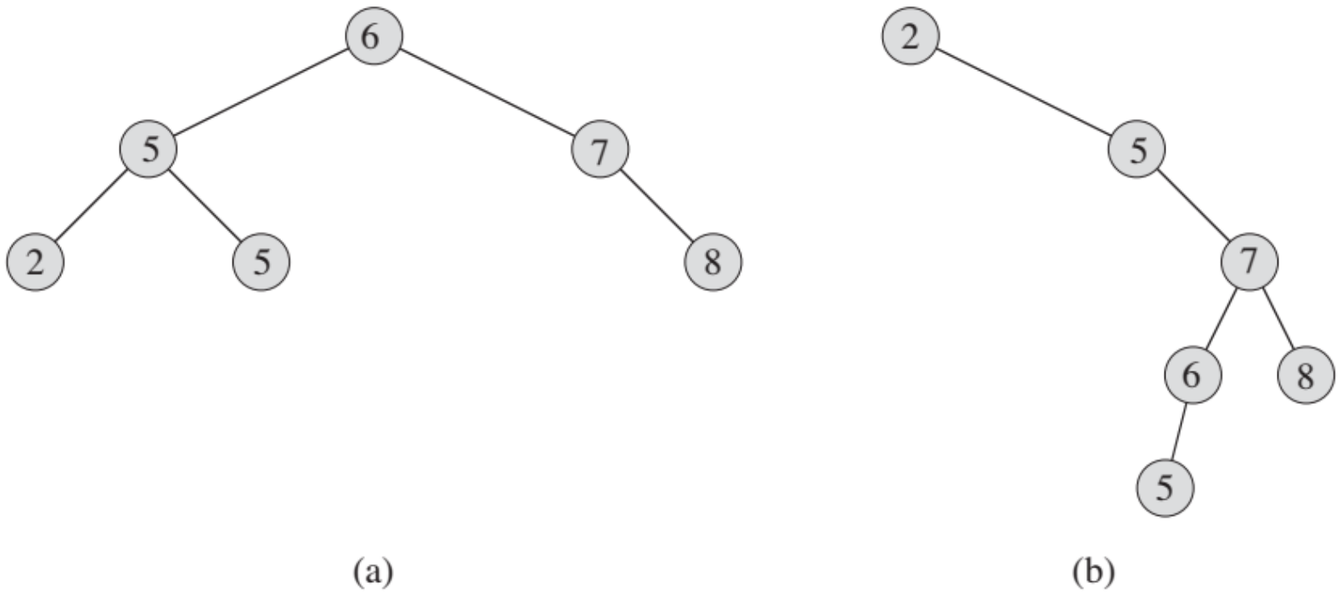


Figure 12.1, CLRS 3rd Edition

Why use BST?

BST is designed to allow fast search operations.

For example, we can use a BST to store the latest transcripts of the students. Each node of the BST would have the roll number of a student as the key and a pointer to the PDF file of the student's latest transcript. When a student attempts to access his/her latest transcript, all there is to do is to search for the student's roll number in the BST.

It was independently invented by different people in the late 1950s.

The Structure of a BST Node

key	left (a pointer to the left child)	right (a pointer to the right child)	p (a pointer to the parent node. For the root node, it is NULL.)	Satellite data (optional)	More attributes (optional)

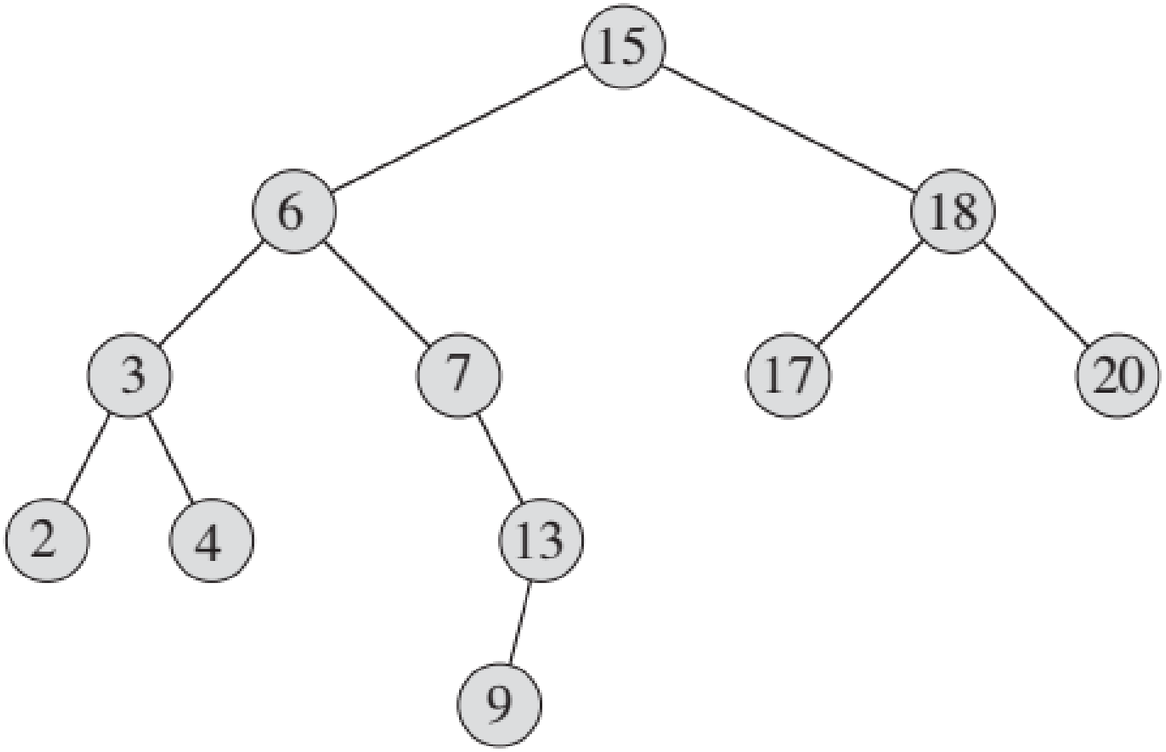
Searching a BST

Input Parameters:

x = The root node.

k = The key to search for.

```
while x ≠ NIL and k ≠ x.key
        if k < x.key
                x = x.left
        else x = x.right
return x
```



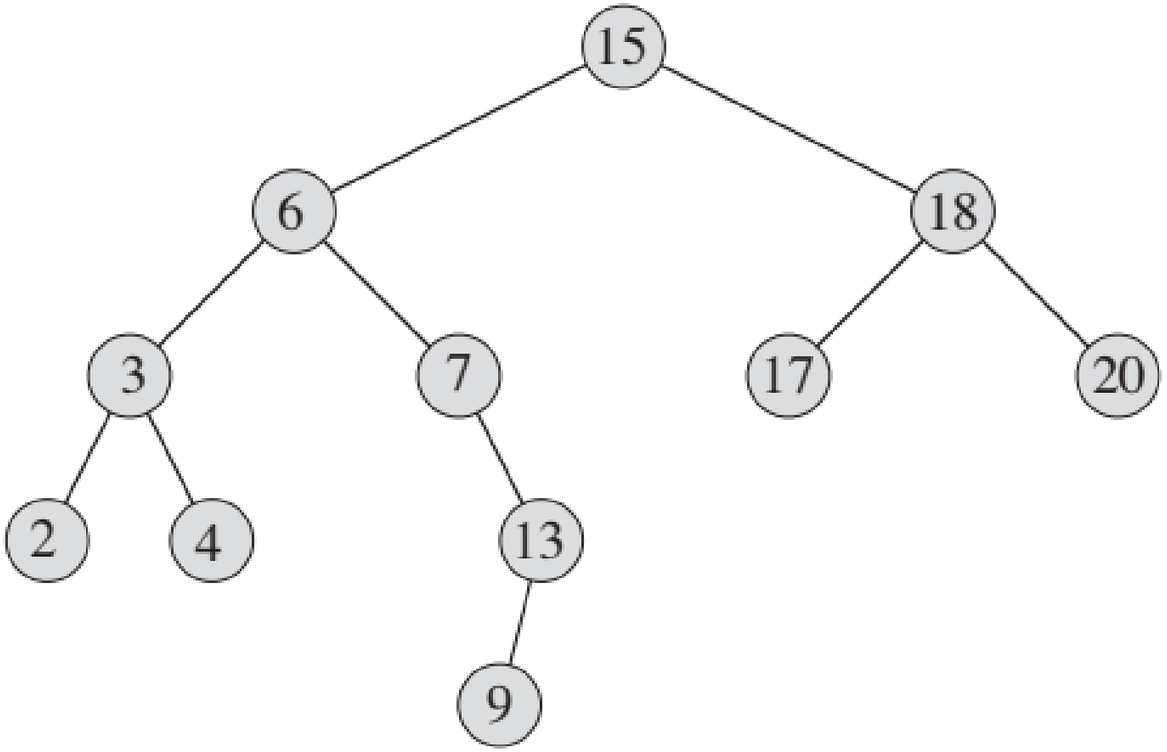
Searching a BST: An Example

Input Parameters:

x = The root node.

k = The key to search for = 13.

```
while x ≠ NIL and k ≠ x.key
        if k < x.key
                x = x:left
        else x = x.right
return x
```



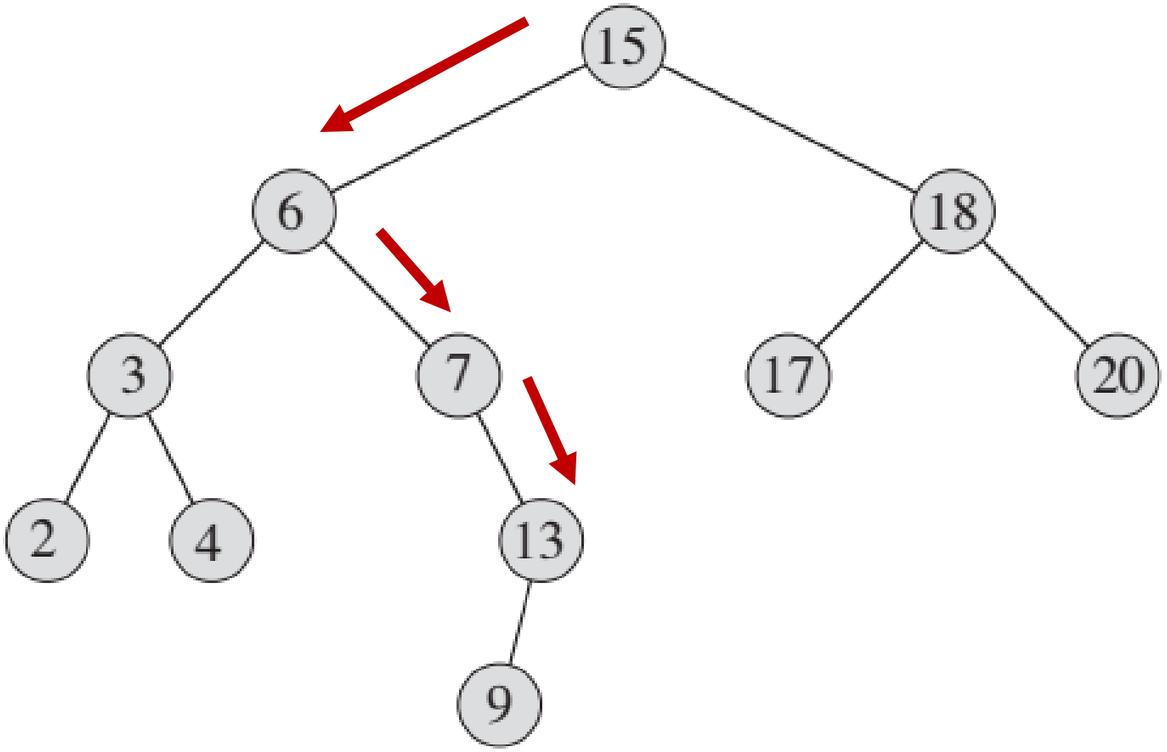
Searching a BST: An Example

Input Parameters:

x = The root node.

k = The key to search for = 13.

```
while x ≠ NIL and k ≠ x.key
        if k < x.key
                x = x:left
        else x = x.right
return x
```



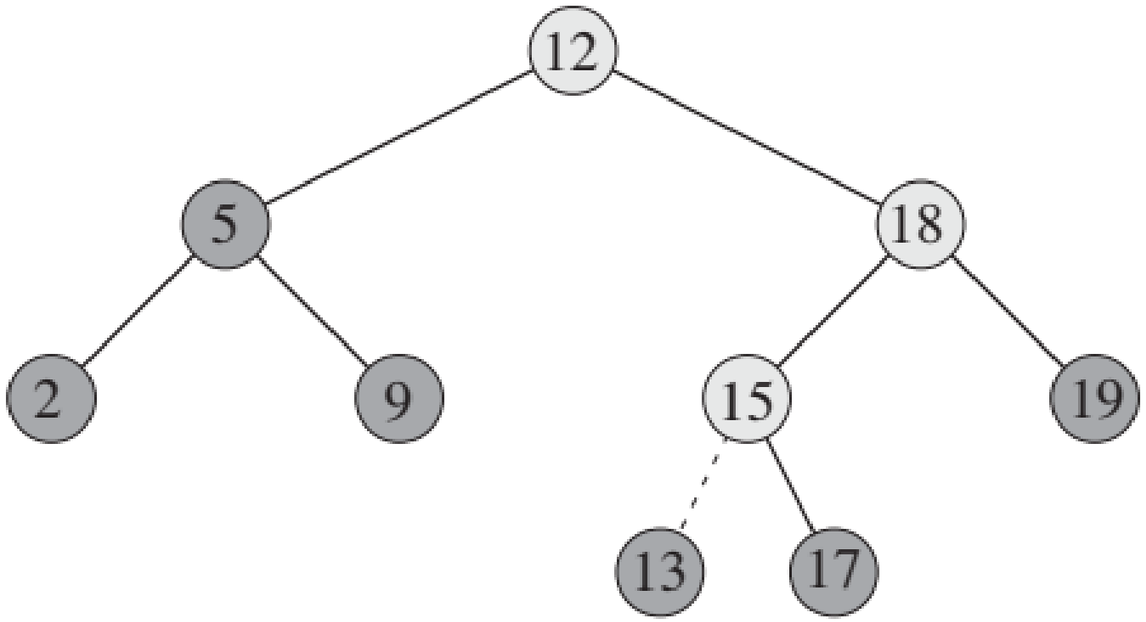
Inserting a New Node into a BST

Input Parameters:

T = The given BST.

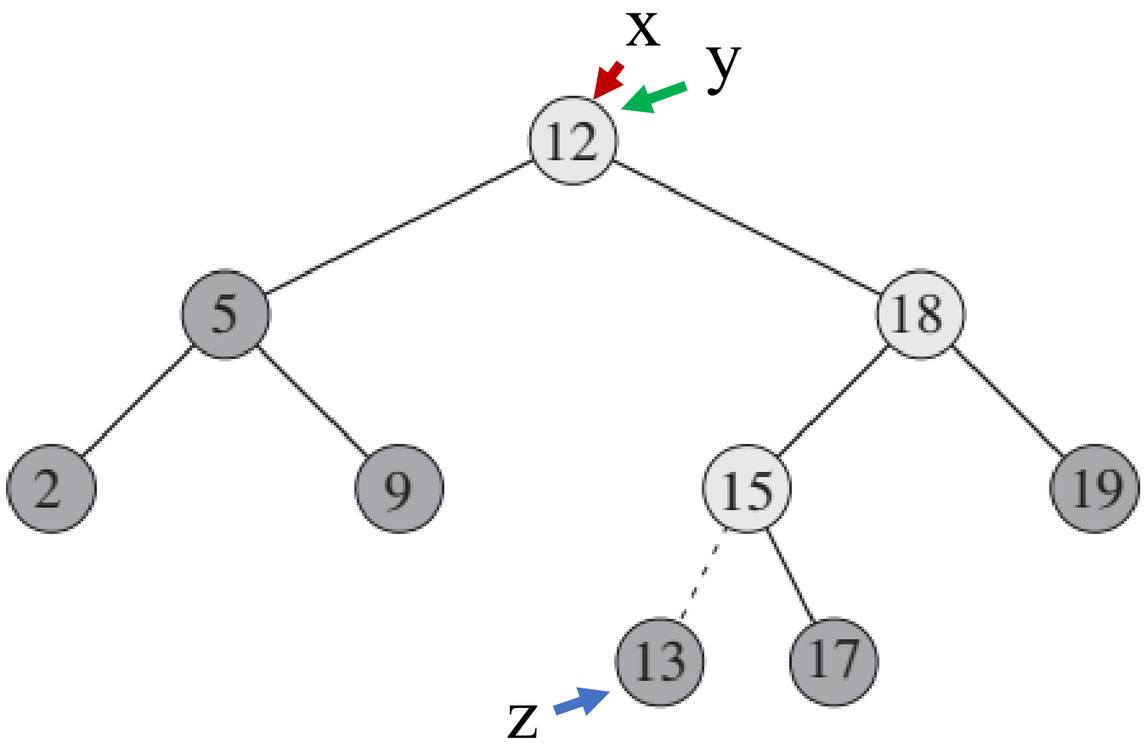
z = The new node to insert

= The node with the key 13 (suppose)



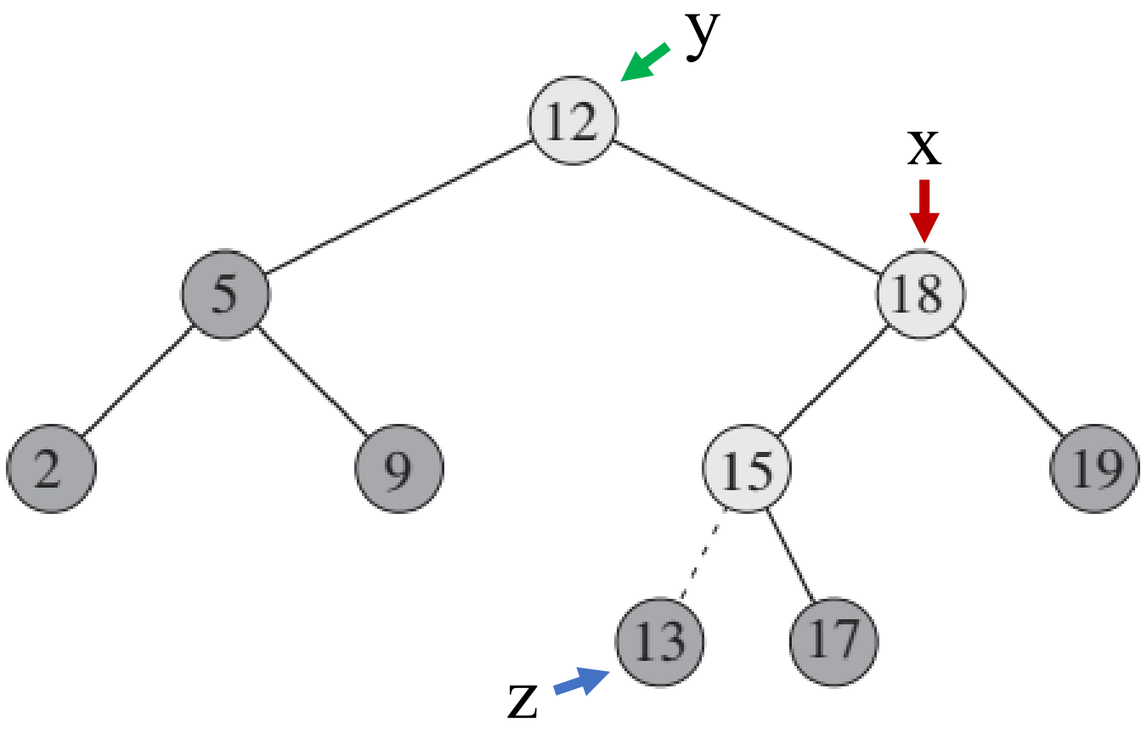
Inserting a New Node into a BST (contd.)

```
y = NIL
x = T.root

while x ≠ NIL
        y = x
        if z.key < x.key
                x = x.left
        else
                x = x.right
```



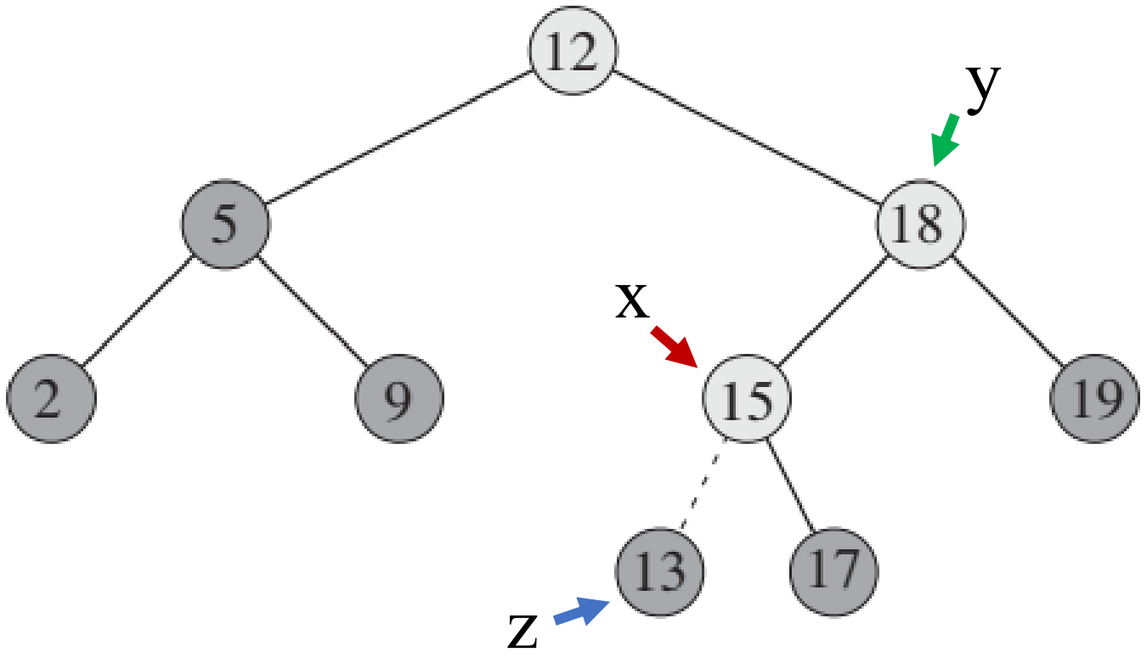
Inserting a New Node into a BST (contd.)

```
y = NIL
x = T.root

while x ≠ NIL
        y = x
        if z.key < x.key
                x = x.left
        else
                x = x.right
```



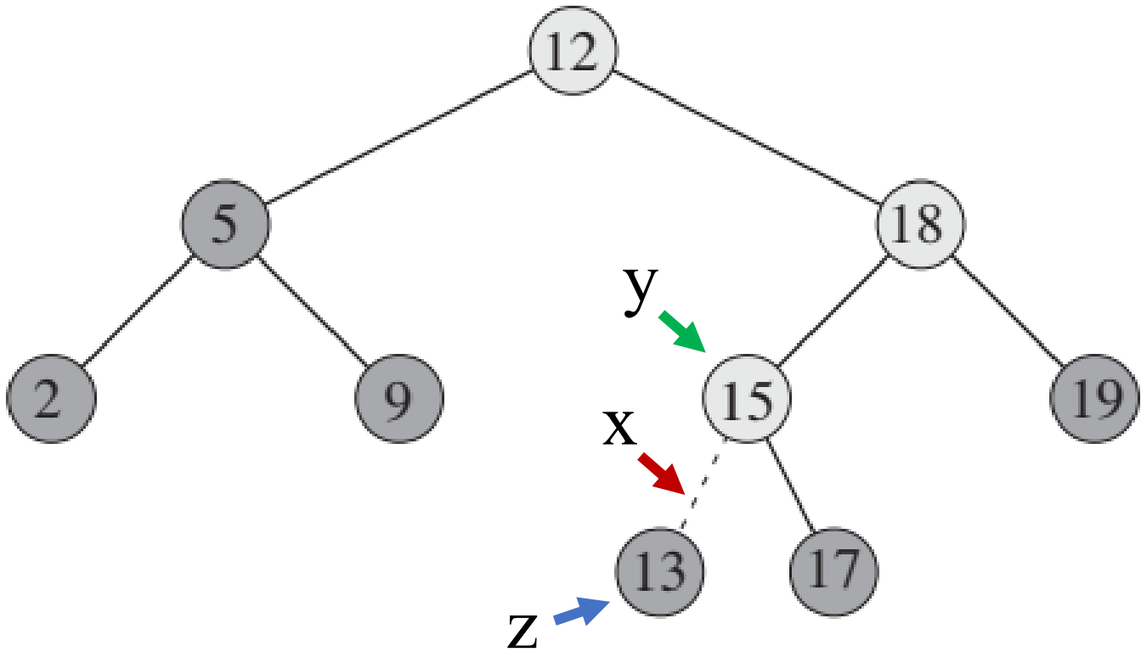
Inserting a New Node into a BST (contd.)

```
y = NIL
x = T.root

while x ≠ NIL
        y = x
        if z.key < x.key
                x = x.left
        else
                x = x.right
```



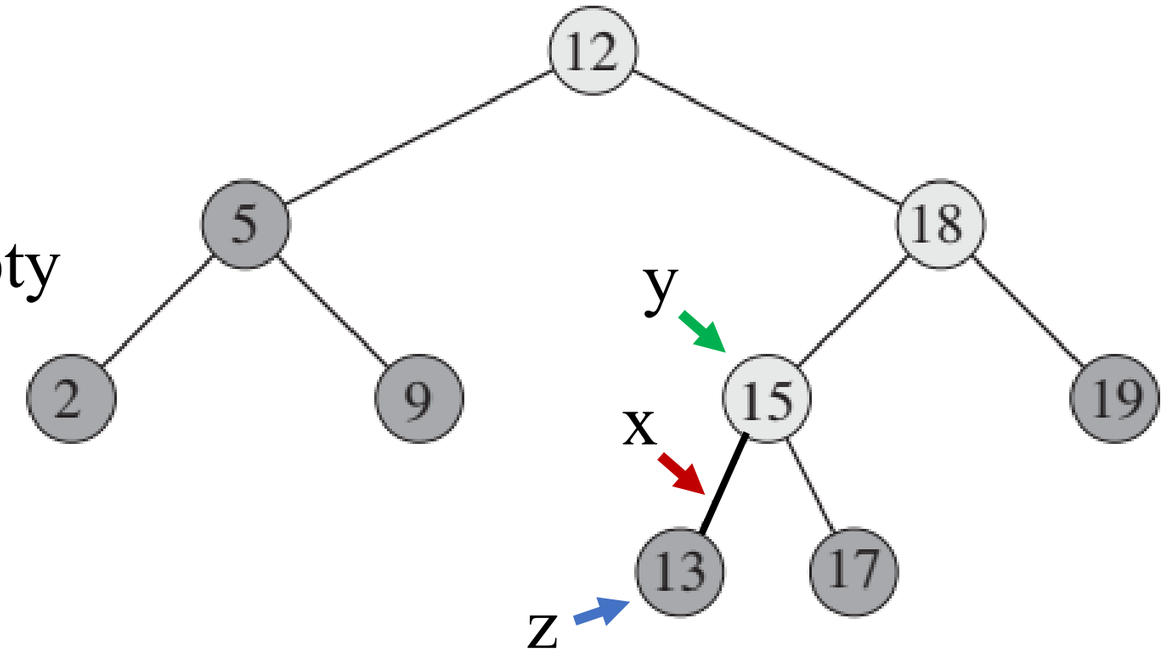
Inserting a New Node into a BST (contd.)

```
y = NIL
x = T.root

while x ≠ NIL
        y = x
        if z.key < x.key
                x = x.left
        else
                x = x.right
```



Inserting a New Node into a BST (contd.)

```
z.p = y
if y == NIL
        T.root = z // Tree T was empty
Else if z.key < y.key
        y.left = z
Else y.right = z
```



Deleting a Node from a BST

We need to define a helper function called TRANSPLANT(T, u, v).

Input Parameters of TRANSPLANT():

T = The given BST.

u , v = The subtree rooted at node u to be replaced with the subtree rooted at node v.

Deleting a Node from a BST (contd.)

```
If u.p == NIL
        T.root = v
Else if u == u.p.left
        u.p.left = v
Else
        u.p.right = v

If v ≠ NIL // v = NIL implies replacing u with NIL
        v.p = u.p
```

Deleting a Node from a BST (contd.)

Input Parameters:

T = The given BST.

z = The node to delete.

Deleting a Node from a BST (contd.)

```
If z.left == NIL
        TRANSPLANT(T, z, z.right)
Else if z.right == NIL
        TRANSPLANT(T, z, z.left)
```

Deleting a Node from a BST (contd.)

```
Else  y = TREE-MINIMUM(z.right)
        If y.p ≠ z
                TRANSPLANT(T, y, y.right)
                y.right = z.right
                y.right.p = y

        TRANSPLANT(T, z, y)
        y.left = z.left
        y.left.p = y
```

Summary

BST allows us to perform fast search operation at the cost of complex insert and delete operations.

In BST, the insert and delete operations become complex because we have to make sure the BST property holds after every insertion or deletion.

Practice Problem

Task 1.

Write a BST insert operation in C and insert the following numbers in the given order into a BST: 8, 18, 1, 13, 24, 7, 0, 5, 9.

Task 2.

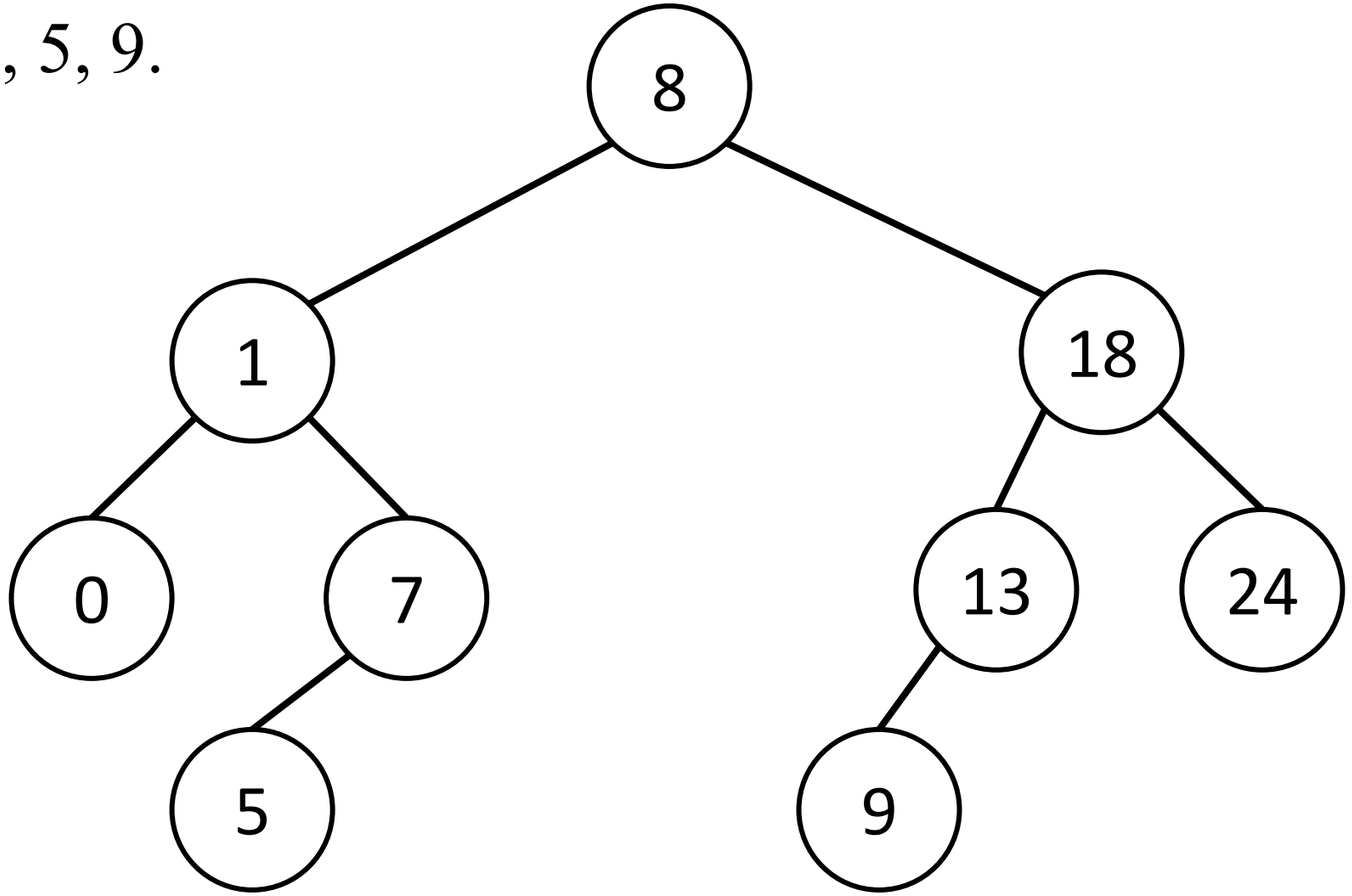
Write a BST search operation in C and search for 0.

Task 3.

Write a BST delete operation in C and delete 18.

Practice Problem (Hint)

Insert 8, 18, 1, 13, 24, 7, 0, 5, 9.



Thank You